

# TIDE: A Domain-Independent Semantic Contract for Exact Execution Relationships

Gabriel Dolbear  
Independent Researcher  
gabrieldolbear@gmail.com

**Abstract**—Systems often retain *execution records*: evidence about completed work, including which occurrence happened, which data it consumed or produced, and the conditions under which it ran. Such records support origin, ancestry, and downstream-inspection questions. Yet two records pertaining to the same work may preserve similar evidence while disagreeing about the exact relationships. To determine what records must distinguish and compare, this paper audits 35 questions from the Second and Third Provenance Challenges and ML Metadata (MLMD). The audit identifies six recurring requirement types. Producer–Consumer relationship occurs in 25 questions across all three corpora and is selected as a shared structural requirement that does not replace host-specific semantics. Its recurring forms ask which process produced or consumed a data item and which inputs and outputs belong to a process. A scoped review finds that existing approaches can often encode these facts, but admissible records may omit selected answers. It identifies no single formal mechanism for exact cross-representation comparison of all four selected answer forms independently of host semantics. TIDE addresses this derived gap within supplied projections through exact Execution–State input/output incidence, validity rules preventing ambiguity and cyclic ancestry, fixed relationship answers, and comparison under supplied identity maps. A small reference comparator applies the contract to one Dagster run and one Python run emitting OpenLineage events over the same six functions. Their retained native records agree about observed relationships; controlled comparisons distinguish partial scope, exact relationship differences, and invalidity. This evidence shows what one comparator can determine after interpretation and identity correspondence are supplied. It neither establishes native-record truth nor discovers correspondence.

**Index Terms**—execution records, mapped-scope conformance, exact execution relationships, provenance interoperability

## I. INTRODUCTION

Work can be represented by a workflow-management system, a lineage event model, a metadata store, or a portable provenance package, each retaining execution evidence for a different purpose and at a different level of detail [1], [2], [3]. When multiple tools contribute evidence about one result, their provenance may need to be integrated or queried through a common representation [4], [5]. Each record can remain useful for its native purpose, yet an operator investigating a result, a platform team reconciling histories, or a consumer tracing downstream effects may still need to determine whether the represented relationships agree.

Comparison here cannot mean byte-for-byte equality or forcing every record into one native schema. Identifiers, terminology, granularity, and host-specific meaning may legitimately differ across systems [3], [4]. Comparison instead requires a

question that can be asked of both records, correspondence between the occurrences being compared, and a rule for determining whether both records preserve the same answers. Without an explicit target, similar-looking records may conceal a material disagreement, while differently structured records may be rejected despite agreeing on the information that matters.

The paper therefore begins by asking what execution records must distinguish for such comparison to be meaningful. A bounded audit of provenance and execution-metadata questions identifies recurring requirements expressed through the intended uses of those records [6]. A scoped review then follows that requirement into existing approaches, asking whether their admissible records require the selected answers and whether their comparison mechanisms compare those answers across representations.

This turns a broad interoperability concern into precise needs. After the audit and review, the paper proposes TIDE as a contract for stating and comparing the selected relationship answers. The remaining sections define the contract, prove its guarantees, evaluate what one comparator can determine from separately represented evidence, and state the boundaries of those results. The contribution arises directly from the identified audited needs and formally checked answers—TIDE is not a workflow system or provenance vocabulary.

## II. RESEARCH QUESTIONS

**RQ1.** Across the bounded question corpus, what exact answers are requested, and which recurring domain-independent information requirements are needed to produce them?

**RQ2.** Which recurring requirements are selected for formalisation, can existing approaches retain them in their admissible records, and what do their cross-system mechanisms require and compare?

**RQ3.** What contract preserves the selected requirements and defines their validity and exact answer sets?

**RQ4.** What does applying this contract enable one host-independent comparator to determine about separately represented records?

## III. QUESTION AUDIT AND RECURRING REQUIREMENT DECOMPOSITION

### A. Bounded Corpus and Audit Method

The bounded corpus contains nine questions analysed from the Second Provenance Challenge, all four core and

fifteen optional Third Challenge questions, and seven MLMD questions—six from the guide and one from the tutorial [2], [7], [8], [9]. These sources state concrete questions expected of provenance or lineage tools; MLMD adds a modern execution-metadata setting. The corpus closes when every question in those declared sets has been reviewed. The retained audit records the selection rationale, protocol, question mappings, and aggregation [6].

For each question, the audit records: (1) the requested answer; (2) the target, type, and filter conditions already supplied; and (3) the information that must be preserved to produce that answer. Table I gives the six resulting requirement types and their counts. One author performed these classifications; they are not source terminology, and the retained question-level map makes each decision inspectable.

TABLE I  
REQUIREMENT TYPES IDENTIFIED BY THE 35-QUESTION AUDIT. COUNTS  
OVERLAP BECAUSE ONE QUESTION MAY REQUIRE SEVERAL TYPES.

| Requirement type               | Required when                                                                                                     | Count |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|-------|
| Identity                       | exact recorded data items or process occurrences must be distinguishable.                                         | 32    |
| Producer-Consumer relationship | which process produced or consumed a data item, or which data items a process consumed or produced, must be known | 25    |
| Execution Status               | success, failure, completion, or halt state affects the answer                                                    | 9     |
| Ordering/Time                  | time, elapsed duration, or order affects the answer                                                               | 6     |
| Domain Semantics               | domain-specific records, values, joins, queries, effects, or operation meaning must be interpreted                | 9     |
| Annotations and Filtering      | names, types, roles, parameters, metadata, or context select the answer                                           | 23    |

### B. Selected Requirement and Boundary

Each recurring type was checked against three conditions: it had to (1) recur across questions; (2) answer domain-independent questions; and (3) be representable without replacing domain-specific semantics. Producer–Consumer relationship meets all three. It appears in 25 questions across the Second Challenge, Third Challenge, and MLMD, and asks which realised process occurrence produced or consumed which data item.

Other types contribute to differing necessities. Identity distinguishes the participants but does not state their relationship; Annotations and Filtering select participants; Execution Status and Ordering/Time qualify answers; and Domain Semantics supplies source-specific interpretation. They remain necessary where the audit records them, but they are not subjects selected for formalisation here.

Within those 25 questions, the recurring subject-specific wording normalises to four exact answers: which process occurrence produced a selected data item; which process occurrences consumed it; which exact data items a process occurrence consumed; and which exact data items it produced. Those answers require distinguishable realised process identi-

ties, distinguishable realised data-item identities, and directed production and consumption relationships.

The selection does not reduce every audited question to lineage. Questions about causes, consequences, values, failure, timing, parameters, comparison, or planned work also require the other types in Table I. The next section therefore asks one narrower question: do existing approaches both retain these four selected answers and compare them across independently represented records?

## IV. EXISTING FORMALISATIONS AND THE REMAINING GAP

After the audit selected Producer–Consumer relationship, the review applied five fixed questions to each work: (1) can it represent the four primitive answers; (2) can its own admissible or minimum record omit information needed for one of them; (3) does it define a formal comparison between independently represented records; (4) what interpretation or identity alignment must be supplied; and (5) what validity, equivalence, translation, or conformance rules are actually checked? Table II reports those answers. The reviewed set is scoped rather than exhaustive; it compares only the selected requirement and does not rank general provenance expressiveness.

The review found the required facts are widely representable. OPM’s *used* and *was generated by* edges and PROV’s *used* and *wasGeneratedBy* relations directly retain identified process–data incidence [10], [11]. The mediator’s global schema and API and Workflow Run RO-Crate likewise expose execution inputs and outputs when their source mappings or declared record requirements provide them [3], [4]. The review gap is therefore not an inability of existing models to encode the four answers.

Representability, however, is not a completeness guarantee. OPM explicitly defines *was triggered by* as a lossy Process summary: completion can introduce the existence of a joining Artifact but cannot recover its identifier [10]. PROV similarly defines *wasInformedBy* as communication through an unspecified Entity. PROV normalization can infer an existential Entity generated by one Activity and used by the other, but it does not recover the identity of the data occurrence asked for by the four-answer requirement [11], [12]. Other omissions are deliberate rather than defective. CPF permits an export consisting of one Entity and a dictionary reference instead of its complete history [13]; Process Run Crate makes action inputs optional and outputs recommended rather than mandatory [14]; and ProvAbs can replace exact nodes while preserving scoped PROV validity and avoiding unjustified new relations [15]. Thus admissible records need not preserve complete selected answers.

The reviewed works also provide substantial, but differently targeted, comparison machinery. OPM defines structural equality, persistent cross-graph names, profiles, and refinement over inferred multistep dependencies [10]. PROV-CONSTRAINTS goes further: it defines validity and equivalence of PROV instances through normalization and isomorphism of valid normal forms [12]. CPF proposes source identifiers, naming dictionaries, and Schematron enforcement for its additional

TABLE II

SCOPED REVIEW AGAINST THE SELECTED FOUR-ANSWER PRODUCER–CONSUMER REQUIREMENT. “ADMISSIBLE” USES EACH WORK’S OWN ACCEPTANCE CRITERION.

| Work                            | Can retain the four answers?                                                          | Does an admissible record require them?                                                                                                                    | Cross-record mechanism                                                                              | Supplied boundary and actual check                                                                                                               |
|---------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| OPM [10]                        | Yes: <i>used</i> and <i>was generated by</i> identify Process–Artifact incidence.     | No. A legal graph may retain lossy <i>was triggered by</i> ; the hidden Artifact exists but its identity is unavailable.                                   | Structural equality, persistent names, profiles, and refinement by inferred multistep dependencies. | Profile conventions or cross-graph names are supplied. These mechanisms preserve or infer OPM structure, not four-answer completeness.           |
| PROV [11], [12]                 | Yes: <i>used</i> and <i>wasGeneratedBy</i> identify Activity–Entity incidence.        | No. <i>wasInformedBy</i> permits communication through an unspecified Entity; normalization supplies an existential, not its recorded occurrence identity. | Validity and equivalence by normalization and isomorphism of valid normal forms.                    | Applications resolve URI co-reference first. Equivalence checks complete PROV normal-form information, not selected four-answer completeness.    |
| Mediator [4]                    | Yes, when wrappers expose execution–data relationships in the global schema.          | No source-independent completeness rule; native omissions and representational choices are resolved by each wrapper.                                       | One mediated schema with common local and transitive query operations.                              | Wrapper authors supply native semantics and identifiers. The case study demonstrates common queries, not pairwise answer equivalence.            |
| CPF [13]                        | Yes, when the exchanged OPM graph includes the relevant typed and named entities.     | No. A single exported entity plus a dictionary reference may be sufficient; complete history need not be transmitted.                                      | Source identifiers, naming dictionary, and proposed Schematron enforcement.                         | Dictionary mappings are supplied. These checks target CPF constraints, not four-answer equality.                                                 |
| ProvONE/P-PIF [5]               | Yes: ProvONE can represent prospective and retrospective workflow relationships.      | Dependent on the exported retrospective trace; the reviewed proof does not require a complete four-answer run record.                                      | A common ProvONE graph and SPARQL interface; a proved Prov2ONE translation algorithm.               | Vocabulary rules and adapters are supplied. The proof concerns prospective workflow translation, not pairwise retrospective-answer preservation. |
| Workflow Run RO-Crate [3], [14] | Yes, through <i>object/result</i> links and optional step detail.                     | No. In Process Run Crate, <i>object</i> is MAY and <i>result</i> is SHOULD.                                                                                | Shared packaging and run profiles support heterogeneous consumption and comparison.                 | Native identifiers and record interpretation remain supplied. Its conformance checks declared requirements, not pairwise answer equality.        |
| ProvAbs [15]                    | The source may; an abstract output may replace exact nodes and remove internal edges. | No. Deliberate abstraction can remove exact identities while retaining a valid PROV graph without unjustified new relations.                               | A formal grouping transformation with source sets and a policy.                                     | The grouping set and policy are supplied. Rules preserve scoped validity and relation soundness, not answer reflection.                          |

constraints [13]; mediation supplies one query interface over source wrappers [4]; P-PIF proves correctness and completeness of its prospective Prov2ONE translation and provides a shared ProvONE query representation [5]; and Workflow Run RO-Crate reports comparison across heterogeneous systems [3]. These mechanisms check structural equality, full-model equivalence, translation, declared requirements, or common query access. They do not state the same bounded obligation as complete preservation and reflection of the four selected answers across two native records.

Nor do they remove the external alignment problem. PROV equivalence assumes that applications have already determined URI co-reference [12]. CPF supplies source identifiers and naming dictionaries, while the mediator requires wrapper authors to decide how native fields, data items, and identifiers populate the global schema [4], [13]. These are necessary host- or domain-dependent premises. A shared relationship contract cannot establish that two native identities really denote corresponding occurrences.

The reviewed set revealed two main obligations for representing and comparing Producer–Consumer relationships. First, a formal contract for preserving and representing the recorded relationships. Second, a formal method for comparison of these relationships under user-supplied identity maps. This does not imply that prior models cannot encode the facts or that their broader mechanisms are deficient; a suitably configured validator could enforce the same bounded conditions. The next section defines TIDE as one precise contract for this obligation.

## V. TIDE EXACT-RELATIONSHIP CONTRACT

### A. Candidate Structure and Identities

A candidate TIDE structure is

$$K = (I, O), \quad I, O \subseteq \mathcal{E} \times \mathcal{S}, \quad (1)$$

where  $\mathcal{E}$  and  $\mathcal{S}$  supply possible Execution and State identities. A realised *Execution* is one identified occurrence of work, computational or otherwise, not a reusable job or operation type. A represented *State* is one identified data or condition occurrence selected as an input or output; State identity is not value equality. Execution and State identifiers are opaque occurrence identities supplied by the host: TIDE neither prescribes a naming syntax nor infers semantic identity from identifier form. A pair  $(e, s) \in I$  means that  $e$  consumed  $s$ , and  $(e, s) \in O$  means that  $e$  produced  $s$ . Membership in  $I$  and  $O$  directly answers the primitive producer, consumer, input, and output questions derived in Section III.

The identities present in  $K$  are

$$E_K = \{e \mid \exists s : (e, s) \in I \cup O\}, \quad (2)$$

$$S_K = \{s \mid \exists e : (e, s) \in I \cup O\}. \quad (3)$$

### B. Fixed Relationship Judgments

The notation  $K \vdash J$  means that judgment  $J$  follows from  $K$ .

*Direct handoff.*

$$\frac{(e_p, s) \in O \quad (e_c, s) \in I}{K \vdash e_p \xrightarrow{s} e_c}. \quad (4)$$

*Directed reachability.*

$$\frac{n \geq 1, \quad e_0 = e_a, \quad e_n = e_b, \quad \forall i < n : K \vdash e_i \xrightarrow{s_i} e_{i+1}}{K \vdash e_a \rightsquigarrow e_b}. \quad (5)$$

These judgments are defined for any fixed  $K$ . Direct handoff relates two Executions through their exact shared State. For example, if  $e_0$  produced  $s_0$  and  $e_1$  consumed  $s_0$ , then  $s_0$  directly connects  $e_0$  to  $e_1$ . Reachability is the non-empty transitive closure of direct handoff. If  $e_1$  then produced  $s_1$  consumed by  $e_2$ , the rules additionally derive that  $e_0$  reaches  $e_2$ . Both judgments are composed completely from  $I$  and  $O$ . They identify structurally upstream and downstream Executions without asserting timing, physical transfer, value-level causal necessity, or capture completeness.

### C. Validity and Ancestry Meaning

A valid  $K$  satisfies two semantic conditions.

*Unique creation.* Every represented State has exactly one represented producer:

$$\forall s \in S_K, \exists! e \in E_K : (e, s) \in O. \quad (6)$$

*Acyclic ancestry.* No Execution reaches itself:

$$\forall e \in E_K, \quad K \not\models e \rightsquigarrow e. \quad (7)$$

These rules reject producerless or multiply produced State occurrences and cyclic handoff structure. Valid records therefore have one represented producer per State and permit reachability to carry TIDE's intended acyclic-ancestry meaning.

Validity remains separate from determination and conformance. Fixed  $I$  and  $O$  still determine answers when a candidate is invalid, and the comparison below can still determine whether mapped incidence corresponds. Only a valid structure, however, carries TIDE's intended unique-production and acyclic-ancestry meaning.

### D. Supplied Interpretation and Mapped-Scope Conformance

A native record is not itself a TIDE structure. A supplied host interpretation selects native evidence and projects it as realised input and output facts; all other fields and domain meanings remain native. The paper assumes that each supplied interpretation correctly handles its selected evidence. TIDE neither discovers this interpretation nor proves the native record true or complete. It begins after two structures and their occurrence correspondence have been supplied; every exactness claim is relative to the incidence in those projections.

Let  $K_1 = (I_1, O_1)$  and  $K_2 = (I_2, O_2)$ . Two supplied bijections represent the identities declared to correspond in the ambient identity spaces:

$$\begin{aligned} f_E : D_E &\xrightarrow{\sim} R_E, & D_E, R_E &\subseteq \mathcal{E}, \\ f_S : D_S &\xrightarrow{\sim} R_S, & D_S, R_S &\subseteq \mathcal{S}. \end{aligned} \quad (8)$$

The maps are not required to associate every identity in either carrier, and their endpoints need not occur in either carrier. Empty maps make no relationship claim about either non-empty carrier. Intersecting the supplied domains and ranges with the record carriers defines the mapped identities present on each side:

$$E_1^m = E_1 \cap D_E, \quad E_2^m = E_2 \cap R_E, \quad (9)$$

$$S_1^m = S_1 \cap D_S, \quad S_2^m = S_2 \cap R_S. \quad (10)$$

Intersection isolates the mapped identities present within a specified representation:

$$I_i^m = I_i \cap (E_i^m \times S_i^m), \quad (11)$$

$$O_i^m = O_i \cap (E_i^m \times S_i^m), \quad i \in \{1, 2\}. \quad (12)$$

Set difference exposes the facts outside the mapped scope:

$$I_i^{\text{out}} = I_i \setminus I_i^m = I_i \setminus (E_i^m \times S_i^m), \quad (13)$$

$$O_i^{\text{out}} = O_i \setminus O_i^m = O_i \setminus (E_i^m \times S_i^m). \quad (14)$$

Thus an outside fact has an unmapped Execution, an unmapped State, or both. It remains visible, but cannot be compared under maps that supply no correspondence for at least one endpoint.

The pair map

$$F : D_E \times D_S \xrightarrow{\sim} R_E \times R_S, \quad F(e, s) = (f_E(e), f_S(s)) \quad (15)$$

associates pairs over the first supplied identity sets with pairs over the second and extends elementwise to sets. The structures conform inside a supplied map exactly when

$$F(I_1^m) = I_2^m, \quad F(O_1^m) = O_2^m. \quad (16)$$

Two equality directions have distinct meanings. Preservation failures are facts required by the first scope but missing from the second; reflection failures are facts present in the second scope but unsupported by the first:

$$\Delta_I^- = F(I_1^m) \setminus I_2^m, \quad \Delta_I^+ = I_2^m \setminus F(I_1^m), \quad (17)$$

$$\Delta_O^- = F(O_1^m) \setminus O_2^m, \quad \Delta_O^+ = O_2^m \setminus F(O_1^m). \quad (18)$$

Conformance holds exactly when all four difference sets are empty. The outside relations are reported separately and do not alter that result.

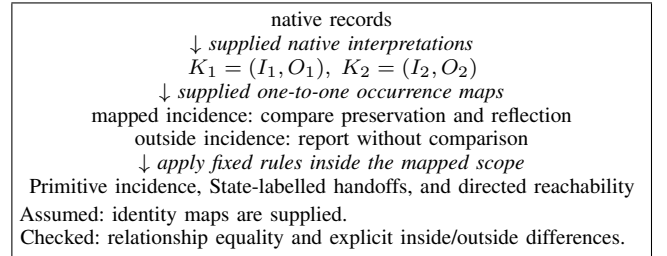


Fig. 1. Interpretation and mapped-comparison boundary.

The supplied maps alone define the comparison scope; neither structure has a privileged role beyond the direction in which facts are renamed. The same rule gives whole-kernel equality when (16) holds and the supplied identity sets cover both record carriers:

$$E_1 \subseteq D_E, \quad E_2 \subseteq R_E, \quad S_1 \subseteq D_S, \quad S_2 \subseteq R_S. \quad (19)$$

All four outside relations are then empty, and the conformant mapped substructures are the complete structures. Extra supplied identity pairs absent from the records do not prevent complete coverage.

Either structure may contain facts outside the declared map scope and still be conformant. Conformance is independent of validity. Either structure may satisfy or violate the conditions in Section V-C without changing whether its mapped incidence

is preserved and reflected. Validity is therefore reported as a separate assertion; when required, it preserves TIDE's unique-production and ancestry meaning rather than redefining conformance.

## VI. FORMAL GUARANTEES

The contract now has to establish three claims: fixed incidence determines the selected answers; erasing any retained identity or direction distinction can change those answers; and mapped conformance preserves corresponding answers.

### A. Determination

**Proposition 1** (Determination). For any fixed  $K = (I, O)$ , the relations  $I$  and  $O$  determine every primitive production/consumption answer and the complete direct-handoff and directed-reachability answer sets.

*Proof.* Primitive answers are membership queries in  $I$  or  $O$ . For every  $e_p, e_c \in E_K$  and  $s \in S_K$ ,

$$K \vdash e_p \xrightarrow{s} e_c \iff (e_p, s) \in O \wedge (e_c, s) \in I.$$

Thus  $I, O$  fix every local answer, including absence. Finite non-empty chains of those handoffs fix reachability.  $\square$  Validity separately constrains ancestry meaning.

### B. Bounded Information-Loss Witnesses

Each witness uses two valid histories with different answers that become identical after one specified erasure.

*Execution identity.* Let  $O = \{(n_1, s_1), (n_2, s_2)\}$ . In  $K_1$ , let  $I_1 = \{(v_1, s_1), (v_2, s_2)\}$ ; in  $K_2$ , let  $I_2 = \{(v_2, s_1), (v_1, s_2)\}$ . Then  $K_1 \vdash n_1 \xrightarrow{s_1} v_1$  while  $K_2 \vdash n_1 \xrightarrow{s_1} v_2$ . Replacing  $n_1, n_2$  by  $n$  and  $v_1, v_2$  by  $v$  makes both observations identical.

*State identity.* Let  $O = \{(p_A, s_A), (p_B, s_B)\}$ . In  $K_1$ ,  $I_1 = \{(c, s_A)\}$ ; in  $K_2$ ,  $I_2 = \{(c, s_B)\}$ . Their handoff answers differ. Replacing  $s_A, s_B$  by  $s$  makes the observations identical and gives the merged State two producers, violating validity.

*Production/consumption direction.* Let  $O_1 = \{(e_0, s)\}$ ,  $I_1 = \{(e_1, s)\}$  and  $O_2 = \{(e_1, s)\}$ ,  $I_2 = \{(e_0, s)\}$ . The handoffs have opposite directions. Replacing ordered  $(I, O)$  with undirected  $I \cup O$  makes the observations identical.

**Proposition 2** (Specified erasures do not preserve answers). None of the three displayed erasures preserves the complete direct-handoff and directed-reachability answer sets for all valid structures.

*Proof.* The paired histories are counterexamples for their respective erasures. The result is limited to these erasures and judgments; it does not establish global minimality, uniqueness, or storage minimality.  $\square$

The witnesses explain why the contract retains three distinctions at once: realised Execution identity, represented State identity, and production versus consumption direction. Removing any one can make different exact relationship answers appear identical.

### C. Mapped-Scope Conformance

The final guarantee connects the comparison rule to the answers a comparator receives. Let  $K_i^m = (I_i^m, O_i^m)$  be the substructure induced by the mapped carriers defined in Section V-D.

**Proposition 3** (Mapped conformance preserves and reflects scoped answers). If  $K_1$  and  $K_2$  satisfy (16), then for every  $e_p, e_c, e_a, e_b \in E_1^m$  and  $s \in S_1^m$ ,

$$K_1^m \vdash e_p \xrightarrow{s} e_c \iff K_2^m \vdash f_E(e_p) \xrightarrow{f_S(s)} f_E(e_c), \quad (20)$$

$$K_1^m \vdash e_a \rightsquigarrow e_b \iff K_2^m \vdash f_E(e_a) \rightsquigarrow f_E(e_b). \quad (21)$$

*Proof.* The mapped input/output equalities preserve and reflect every primitive fact after association. A direct handoff is exactly one mapped output fact joined to one mapped input fact on the same State, so the first equivalence follows. Mapping every identity in a finite handoff chain gives a chain in the second mapped substructure; the inverse bijections give the reverse implication and therefore the reachability equivalence.  $\square$

The result concerns answers derived inside the mapped substructures. Outside facts may add other handoffs or paths to either complete structure, but are not compared without a supplied mapping. Validity is not used by the proof and remains a separate assertion.

## VII. EVALUATION: WHAT TIDE ALLOWS A COMPARATOR TO DETERMINE

### A. What Was Used and What Was Tested

After two native records have been projected and identity mapping supplied, the evaluation asks whether one TIDE consumer can determine: whether mapped producer-consumer facts agree; which exact facts differ; what lies outside the map; whether each structure is valid; and whether the map is one-to-one.

There is one six-operation workload. It is executed twice: once by the Dagster runner and once by the Python/OpenLineage runner. Those two runs produce the one retained native-record pair used for cross-system comparison. The remaining comparisons perform no further execution. They use two fixed TIDE structures with different short identities and vary only the supplied map, one relation fact, or one added relation and its required identity pair, isolating each diagnostic scenario. Table III states every expected result.

A standard-library Python comparator accepts finite JSON-compatible  $K = (I, O)$  structures and supplied maps [6]. It checks the structure, derives handoff and reachability, checks validity, and compares mapped incidence. Before comparison, it checks only that both maps are well-formed and one-to-one; mapped identities need not occur in either structure. It then reports absent mapped identities, mapped differences, and outside facts. An existing unmapped identity is outside scope, not an error. Whole-kernel equality additionally requires conformance and carrier coverage on both sides; extra supplied pairs absent from the records do not prevent complete coverage.

TABLE III

EVALUATION CASES AND RESULTS. THE FIRST ROW COMPARES RETAINED RECORDS FROM TWO EXECUTIONS OF THE SAME SIX-OPERATION WORKLOAD. THE REMAINING ROWS BEGIN WITH ONE CONTROLLED PAIR OF SIX-EXECUTION, SEVEN-STATE FIXTURES HAVING THE SAME RELATIONSHIP TOPOLOGY; EACH CASE VARIES ONLY THE SUPPLIED MAP, ONE RELATION FACT, OR ONE ADDED MAPPED RELATION AND ITS REQUIRED IDENTITY PAIR, AND PERFORMS NO ADDITIONAL EXECUTION. ALL ROWS USE THE SAME COMPARATOR.  $\Delta I$  AND  $\Delta O$  REPORT MISSING OR UNSUPPORTED MAPPED FACTS; “OUTSIDE” REPORTS FACTS NOT COVERED BY THE MAPS.

| Comparison                                      | Valid $K_1/K_2$ | $\Delta I$ $-/+$ | $\Delta O$ $-/+$ | Outside $I/O$ $K_1; K_2$ | Conforms | Whole equal | Enabled decision                                    |
|-------------------------------------------------|-----------------|------------------|------------------|--------------------------|----------|-------------|-----------------------------------------------------|
| Dagster/OpenLineage run records                 | Y/Y             | 0/0              | 0/0              | 0/0; 0/0                 | Y        | Y           | complete cross-system agreement                     |
| Controlled structures, full carrier coverage    | Y/Y             | 0/0              | 0/0              | 0/0; 0/0                 | Y        | Y           | matching baseline                                   |
| Controlled structures, partial carrier coverage | Y/Y             | 0/0              | 0/0              | 3/3; 3/3                 | Y        | N           | agreement in supplied scope                         |
| Omitted mapped input                            | Y/Y             | 1/0              | 0/0              | 0/0; 0/0                 | N        | N           | missing consumption fact                            |
| Added mapped-scope input                        | Y/Y             | 0/1              | 0/0              | 0/0; 0/0                 | N        | N           | unsupported consumption fact                        |
| Omitted mapped output                           | Y/N             | 0/0              | 1/0              | 0/0; 0/0                 | N        | N           | missing production; invalid target                  |
| Mapped output, target identity absent           | Y/Y             | 0/0              | 1/0              | 0/0; 0/1                 | N        | N           | absent target identity; expected production missing |
| Added mapped-scope output                       | Y/Y             | 0/0              | 0/1              | 0/0; 0/0                 | N        | N           | unsupported production; absent source identity      |
| Invalid outside branch                          | Y/N             | 0/0              | 0/0              | 0/0; 1/0                 | Y        | N           | validity separate from scope                        |

### B. Comparing Records Produced by the Two Runners

*Evidence and test.* Both runners call the same unchanged six functions retained in `history.py`. They form one fan-in/fan-out workload: two equal-valued but distinct ready States enter merge, whose output is consumed by both `analyse` and `archive`. Dagster 1.13.16 executes them as operations and retains `STEP_OUTPUT/LOADED_INPUT` events. The Python runner executes them again and uses OpenLineage 1.52.0 to retain `COMPLETE` RunEvents. Each native record is projected to  $I, O$ , and the supplied maps cover both projections. The test asks whether every exact input/output fact is preserved and reflected.

*Result.* Each projection contains six Executions, seven States, six input facts, and seven output facts. Both are valid and derive six State-labelled handoffs and thirteen non-empty reachability answers. Every mapped difference and outside set is empty, so the projections conform and are whole-kernel equal.

*What this shows.* With interpretation and correspondence supplied, one consumer can compare exact relationship answers from different record forms. TIDE neither infers the maps nor proves either native record complete or true.

### C. Separating Scope, Relationship Differences, and Invalidity

*Evidence and test.* This group performs no additional execution. Two fixed TIDE structures represent the same six-Execution, seven-State pattern under different short identity sets. Full carrier coverage establishes the matching baseline; each later case changes only the map, one relation fact, or one added relation and its required identity pair.

*Result.* A partial-coverage map covers four Executions and four States: three input and four output facts agree inside, while each structure has three input and three output facts outside. It conforms but is not whole equal. Removing a mapped input reports  $|\Delta_I^-| = 1$ ; adding one reports  $|\Delta_I^+| = 1$ ; and removing a mapped output reports  $|\Delta_O^-| = 1$  and separately leaves its consumed State producerless. Mapping the source output to an identity absent from the target remains assessable, reports the absent target identity and  $|\Delta_O^-| = 1$ , and leaves the

target’s differently identified output outside scope. Adding a mapped-scope output instead reports an absent source identity and  $|\Delta_O^+| = 1$  while both structures remain valid. Adding a producerless branch outside the map makes the target invalid while mapped differences remain empty.

*What this shows.* Unmapped evidence, mapped disagreement, and invalidity are different results: outside facts are not compared, mapped differences name the exact missing or unsupported fact, and validity is judged separately.

### D. Reproduction and Overall Finding

The artifact retains the comparator, fourteen tests—five comparator unit tests and nine evaluation-table tests—fixed cases, raw native events, projections, maps, expected counts, and audit materials [6]. The runners reproduce the first row of Table III; controlled cases reproduce the rest.

Together, the results show what TIDE adds after projection: one comparator can compare exact mapped relationships, locate disagreement, retain unmatched evidence, and keep conformance, validity, and complete coverage distinct. Proposition 3 guarantees corresponding handoff and reachability answers inside a conforming scope. The evidence establishes evaluation fidelity to this bounded contract, not the truth of supplied interpretation or correspondence.

## VIII. DISCUSSION AND LIMITATIONS

**RQ1.** The audit identifies six recurring requirement types: Identity in 32 questions, Producer–Consumer relationship in 25, Annotations and Filtering in 23, Execution Status in 9, Domain Semantics in 9, and Ordering/Time in 6. Producer–Consumer relationship is the common structural requirement selected for this paper.

**RQ2.** The reviewed approaches can represent its four exact answers, but their admissible records do not uniformly require those answers and their comparison mechanisms target broader or different obligations. Native interpretation and identity co-reference also remain external. The remaining bounded need is therefore a rule that compares exact selected incidence

under supplied identity correspondence and exposes what that correspondence does not cover.

**RQ3.** TIDE supplies that rule. *I* and *O* state and determine the exact relationship answers. One-to-one Execution and State maps choose the comparison scope; preservation and reflection compare incidence inside it; set difference reports incidence outside it; and conformance with complete carrier coverage gives whole-kernel equality. Unique production and acyclic ancestry remain separate validity conditions.

**RQ4.** A single comparator turns those definitions into separate decisions: whether supplied maps are one-to-one, whether mapped facts agree, which exact facts differ, which facts lie outside scope, and whether each structure is valid. The retained native pair demonstrates complete cross-system agreement; the controlled cases demonstrate that partial scope, relationship disagreement, invalidity, and complete coverage are not the same result.

#### A. Scope and Evidence Boundaries

A host-dependent interpretation determines which native records are selected, how they become input/output facts, how Execution and State occurrences are identified, and which occurrences correspond across hosts. The comparator checks map-entry form and one-to-one pairing but treats carrier membership as comparison evidence; it cannot establish semantic co-reference. Incidence and paths outside the mapped scope are reported but not compared. The set-based contract does not represent repeated or port-specific incidence, and its bijections do not express many-to-one granularity alignment. Structural upstream reachability identifies ancestry candidates, not causal necessity; downstream closure identifies potentially affected descendants, not every realised consequence. Domain independence is a property of the contract's types and rules; the present controlled evidence remains workflow-specific and does not establish deployment across multiple domains. The artifact does not measure defect prevalence, discover mappings automatically, measure deployment performance, or show that every host record can be projected into TIDE. TIDE complements rather than replaces richer provenance models; additional answer dimensions require their own semantics and preservation contracts.

### IX. CONCLUSION

After audit and review, this paper asks one bounded question: after two execution records have been independently represented, can one comparator determine whether they preserve the same exact producer, consumer, input, and output answers? The question audit finds that relationship requirement in 25 of 35 questions and across all three corpora. The scoped review shows that existing approaches can encode the facts, but their admissible records may omit selected answers and their comparison mechanisms address other obligations.

TIDE supplies the narrower contract needed at that boundary. Exact Execution–State input/output incidence states the

answers. Supplied maps choose the shared scope. Preservation and reflection locate missing or unsupported mapped facts, while unmatched facts remain visible outside. A conforming scope gives corresponding handoff and reachability answers; conformance with complete carrier coverage gives whole-kernel equality; and validity remains a separate assertion.

The retained Dagster/OpenLineage pair demonstrates one native cross-system comparison. The controlled cases then isolate partial scope, relationship disagreement, and invalidity without additional executions. Together, the contract and evidence show what TIDE enables: one comparator can compare exact execution relationships and locate where supplied projections stop agreeing. TIDE does not discover native interpretation or identity correspondence, and it does not prove native records complete or true. Future work could assess TIDE's utility in agentic workflows and business-process modelling, and its scalability on large execution graphs.

#### ACKNOWLEDGMENT

ChatGPT (OpenAI) [16] supported drafting and restructuring throughout Sections I–IX, source checking, and artifact analysis. The author originated and approved the model, questions, scope, evidence, and conclusions, verified the manuscript, and accepts responsibility for every claim; ChatGPT is not an author.

#### REFERENCES

- [1] OpenLineage, “Object model,” *OpenLineage Core Specification*, accessed Aug. 6, 2026. [Online]. Available: <https://openlineage.io/docs/spec/object-model/>
- [2] TensorFlow, “ML Metadata,” accessed Aug. 2, 2026. [Online]. Available: <https://www.tensorflow.org/tfx/guide/mlmd>
- [3] S. Leo *et al.*, “Recording provenance of workflow runs with RO-Crate,” *PLOS ONE*, vol. 19, no. 9, e0309210, 2024, doi: 10.1371/journal.pone.0309210.
- [4] T. Ellkvist, D. Koop, J. Freire, C. T. Silva, and L. Strömbäck, “Using mediation to achieve provenance interoperability,” in *Proc. IEEE Congress on Services-I*, 2009, pp. 291–298, doi: 10.1109/SERVICES-I.2009.68.
- [5] A. Prabhune, A. Zweig, R. Stotzka, J. Hesser, and M. Gertz, “P-PIF: A ProvONE provenance interoperability framework for analyzing heterogeneous workflow specifications and provenance traces,” *Distrib. Parallel Databases*, vol. 36, no. 1, pp. 219–264, 2018, doi: 10.1007/s10619-017-7216-y.
- [6] G. Dolbear, “TIDE exact-relationship conformance and question-audit artifact,” 2026. [Online]. Available: <https://github.com/tkninobirb/TIDE-Model-Artifact/commit/502ac645ef2fe24ed1fdffa81e49b60c52c8def4>
- [7] Provenance Challenge, “Second Provenance Challenge,” accessed Aug. 3, 2026. [Online]. Available: <https://openprovenance.org/provenance-challenge/SecondProvenanceChallenge.html>
- [8] Provenance Challenge, “Provenance Challenge 3 questions,” accessed Aug. 2, 2026. [Online]. Available: <https://openprovenance.org/provenance-challenge/ProvenanceQuestionsPc3.html>
- [9] TensorFlow, “Better ML engineering with ML Metadata,” accessed Aug. 2, 2026. [Online]. Available: [https://www.tensorflow.org/tfx/tutorials/mlmd/mlmd\\_tutorial](https://www.tensorflow.org/tfx/tutorials/mlmd/mlmd_tutorial)
- [10] L. Moreau *et al.*, “The Open Provenance Model core specification (v1.1),” *Future Gener. Comput. Syst.*, vol. 27, no. 6, pp. 743–756, 2011, doi: 10.1016/j.future.2010.07.005.
- [11] L. Moreau and P. Missier, Eds., “PROV-DM: The PROV data model,” W3C Recommendation, Apr. 2013. [Online]. Available: <https://www.w3.org/TR/prov-dm/>
- [12] J. Cheney, P. Missier, and L. Moreau, “Constraints of the PROV data model,” W3C Recommendation, Apr. 2013. [Online]. Available: <https://www.w3.org/TR/prov-constraints/>
- [13] U. J. Braun, M. I. Seltzer, A. Chapman, B. Blaustein, M. D. Allen, and L. Seligman, “Towards query interoperability: PASSING PLUS,” in *Proc. 2nd USENIX Workshop on the Theory and Practice of Provenance (TaPP)*, 2010. [Online]. Available: <https://www.usenix.org/conference/tapp-10/towards-query-interoperability-passing-plus>
- [14] Workflow Run RO-Crate Working Group, “Process Run Crate, version 0.5,” accessed Aug. 5, 2026. [Online]. Available: <https://w3id.org/ro/wfrun/process/0.5>
- [15] P. Missier, J. Bryans, C. Gamble, V. Curcin, and R. Danger, “ProvAbs: Model, policy, and tooling for abstracting PROV graphs,” in *Provenance and Annotation of Data and Processes*, 2015, pp. 3–15, doi: 10.1007/978-3-319-16462-5\_1.
- [16] OpenAI, “ChatGPT,” accessed Aug. 7, 2026. [Online]. Available: <https://chatgpt.com/>